El desarrollo del frontend utilizando Desarrollo Basado en Especificaciones (SDD) requiere pasar de un contrato de API (backend) a un contrato de **UI/UX y Manejo de Estado**.

Al guiar a un agente de IA en el frontend, la tentación de la herramienta es generar componentes gigantes con lógica visual y llamadas `fetch` mezcladas. Para evitar esto, el enfoque SDD en la capa de usuario debe estructurarse mediante las siguientes buenas prácticas y recomendaciones.

1. Configuración del Contexto Global en `AGENTS.md` (Frontend Rules)

Al igual que en el backend, el agente de IA debe leer primero la arquitectura del cliente. Define explícitamente en el archivo de reglas del proyecto:

- **Stack Fijo:** React 18+, Vite, React Router DOM v6+, y gestor de estado/fetches (ej. React Query / TanStack Query o Axios + Context API). <<-- Revisar el Stack definido para el backend

- **Patrón de Componentes:** Separación estricta entre **Presentacionales** (componentes tontos que solo reciben props y renderizan UI) y **Contenedores/Páginas** (manejan lógica, llamadas a API y estado).

- **Estructura de Directorios:**

```
src/
├── api/          # Funciones aisladas de llamada a los endpoints REST
├── components/   # Componentes reutilizables de UI (Button, Modal, Input)
├── features/     # Módulos por dominio (boards, columns, tickets)
├── hooks/        # Custom hooks para lógica reusable (useAuth, useFetch)
├── pages/        # Vistas asociadas a rutas de React Router
└── routes/       # Configuración de React Router DOM
```

2. Especificación de Contratos de UI y Rutas (SPA Architecture)
Antes de generar vistas, debes entregarle al agente la especificación de navegación y jerarquía de rutas para la Single Page Application (SPA).

- **Matriz de Rutas:** Provee una tabla clara con el mapeo de URL, componentes y estados de protección:
-

Ruta	Componente	Acceso	Descripción / Requerimiento
/login	LoginPage	Público	Formulario de autenticación.
/boards	BoardsListPage	Protegido	Lista de tableros del usuario.
/boards/:boardId	BoardDetailPage	Protegido	Vista principal Kanban. Carga columnas y tickets de forma anidada.

- **Límite Negativo (React Router):** *"NO utilices componentes* <Link> *para acciones de mutación (crear, borrar, mover tickets). Utiliza navegación programática (*useNavigate*) únicamente después de que la mutación en la API haya sido exitosa."*

3. Especificación para Formulario y Mantenimiento de Estado (Forms & Validations)

Los formularios son propensos a generar código redundante cuando se delegan a una IA. La especificación debe cubrir:

1. **Validación en Cliente vs. Servidor:**
    - Exigir el uso de librerías ligeras como **React Hook Form** junto con esquemas de validación (ej. **Zod** o **Yup**).
    - **Especificación de UI:** Indicar explícitamente que los errores devueltos por el backend (ej. 400 Bad Request por título duplicado) deben mapearse directamente a las alertas visuales de los inputs del formulario sin recargar la página.

    ○
2. **Manejo de Estados de Carga (UX Feedback):**
    ○ Fijar la regla: *"Todo formulario que realice un* `POST`*,* `PATCH` *o* `DELETE` *debe deshabilitar el botón de envío (*`disabled={isSubmitting}`*) y mostrar un indicador visual (spinner) mientras la petición HTTP esté en curso."*

4. Partición Generativa en el Frontend (Fases SDD)

Aplica el flujo modular para evitar que la IA pierda el hilo o genere estados inmanejables:

Fase 1: Capa de Servicio API (`src/api`)

Pídele al agente que escriba únicamente los métodos de integración con los endpoints del backend que ya desarrollaron.

Prompt Modelo: *"Basándote en la especificación de endpoints REST de DevBoard (*`POST /api/boards`*,* `GET /api/boards/:boardId`*, etc.), genera un módulo cliente en* `src/api/boardService.js` *utilizando Axios. Incluye manejo de errores y propagación de excepciones."*

Fase 2: Componentes Atómicos de UI

Genera la biblioteca de componentes dummy (tontos) sin conexión a API.

Prompt Modelo: *"Crea un componente* `TicketCard.jsx` *que reciba por props un objeto* `ticket` *(*`{ title, description, priority }`*) y un callback* `onDelete`*. Aplica estilos básicos y asegúrate de que no contenga ningún* `useEffect` *ni llamada a API interna."*

Fase 3: Integración de Vistas y Rutas

Ensambla las páginas conectando los servicios con los componentes mediante `React Router DOM`.

Prompt Modelo: *"Crea la vista* `BoardDetailPage.jsx` *asociada a la ruta* `/boards/:boardId`*. Usa* `useParams` *para obtener el ID, consume* `boardService.getBoardById` *al montar el componente, e implementa estados de* `loading`*,* `error` *y* `data`*."*

5. Control de Concurrencia Visual y Sincronización (Optimistic UI)

En aplicaciones tipo Kanban, la experiencia de usuario depende de la fluidez al arrastrar o mover tickets entre columnas (`PATCH /api/boards/:boardId/columns/:columnId/tickets/:ticketId`).

- **Instrucción SDD para la IA:** Especifica si las mutaciones deben ser **Optimistas** (actualizar la interfaz antes de que el servidor responda y revertir si falla) o **Petición Estándar** (esperar el `200 OK` para refrescar la UI).

- **Regla de Sincronización:** *"Al mover un ticket de columna, aplica actualización optimista en el estado local. Si el backend responde con un error (`400` o `404`), revierte el ticket a su columna original y despliega un mensaje emergente (Toast) notificando el fallo."*

6. Integración con el Workflow de Desarrollo (Vite & Mocking)

Para permitir que los estudiantes desarrollen componentes frontend sin depender 100% de que el servidor local esté corriendo en todo momento:

- **Mock Service Worker (MSW) o Mocks:** Pide a la IA que genere datos sintéticos (`mockData.js`) basados en la especificación del backend.
-
- **Variables de Entorno con Vite:** Definir que el endpoint base de la API debe consumirse exclusivamente desde `import.meta.env.VITE_API_URL` para facilitar el despliegue posterior.

Implementar uno de los siguientes estilos visuales para el look-and-feel de la aplicación.

Pueden pedir al agente que cree su propio CSS, o utilizar Tailwind CSS, Bootstrap u otro.

1. Minimalismo Developer (Estilo IDE/Terminal) Diseñado para reducir la fatiga visual en usuarios técnicos, inspirado en editores de código y herramientas CLI. Prioriza el contraste y la jerarquía de la información por sobre los elementos decorativos excesivos.

- **Light Mode:** Fondo `#F9FAFB` (Gris hielo), Tarjetas `#FFFFFF`, Texto principal `#111827` (Pizarra oscuro), Acento `#4F46E5` (Índigo).
- **Dark Mode:** Fondo `#0F172A` (Azul medianoche), Tarjetas `#1E293B`, Texto principal `#F8FAFC` (Gris muy claro), Acento `#818CF8` (Índigo suave).

2. Modern SaaS (Vibrante y Accesible) Enfocado en herramientas de productividad actuales, utilizando colores primarios llamativos para los Call to Action (CTA) y bordes redondeados suaves para dar una sensación más amigable.

- **Light Mode:** Fondo `#FAFAFA`, Tarjetas `#FFFFFF` con sombras difuminadas, Texto `#2D3748`, Acentos `#00B8D9` (Cian) y `#6554C0` (Púrpura).
- **Dark Mode:** Fondo `#121212`, Tarjetas `#1E1E1E` delimitadas por bordes sutiles `#333333` (sin sombras), Texto `#E0E0E0`, Acentos `#33E1ED` (Cian neón) y `#9F8DF1` (Púrpura claro).

3. Industrial Monocromático Una estética corporativa sobria, ideal para entornos empresariales rígidos que requieren concentración. Utiliza una escala de grises dominante con un único color de acento reservado estrictamente para estados activos y alertas.

- **Light Mode:** Fondo `#FFFFFF`, Tarjetas `#F3F4F6`, Texto `#000000`, Acento principal `#10B981` (Verde esmeralda).
- **Dark Mode:** Fondo `#000000`, Tarjetas `#171717`, Texto `#D4D4D4`, Acento principal `#34D399` (Verde esmeralda brillante).

4. Silicon Valley Retro-Tech Un guiño estético a las interfaces de sistemas informáticos clásicos pero con una experiencia de usuario moderna, utilizando paletas cálidas y tipografías monoespaciadas para los identificadores de tickets.

- **Light Mode:** Fondo `#FDF6E3` (Crema cálido), Tarjetas `#EEE8D5`, Texto `#073642` (Azul petróleo oscuro), Acento principal `#CB4B16` (Naranja ladrillo).
- **Dark Mode:** Fondo `#002B36` (Petróleo muy oscuro), Tarjetas `#073642`, Texto `#839496` (Gris verdoso), Acento principal `#2AA198` (Cian retro).

Requerimientos de Implementación (Theme Switch Light/Dark) Para asegurar que los estudiantes apliquen buenas prácticas en el manejo del tema, la especificación de desarrollo debe exigir los siguientes lineamientos arquitectónicos:

- **Variables CSS (Custom Properties):** Queda estrictamente prohibido "hardcodear" colores hexadecimales directamente en los componentes. Se deben definir tokens globales (`--bg-primary`, `--text-primary`, `--accent-color`) en el nivel de diseño.
- **Detección de Preferencia del Sistema:** La aplicación debe leer inicialmente la media query `@media (prefers-color-scheme: dark)` para adoptar el tema nativo del sistema operativo del usuario sin intervención manual.
- **Gestión y Persistencia de Estado:** La selección explícita del usuario a través del interruptor de la interfaz debe inyectar una clase global (ej. `.dark`) en la etiqueta `<html>` o `<body>`. Esta preferencia debe guardarse en `localStorage` para sobrevivir a los refrescos de página y evitar el "parpadeo" de estilos al cargar el sitio.

Elaborar un archivo en formato .md para guardar todos los requerimientos y luego asegurarse que el agente lo lea y respete.

Actualizar repositorios y desplegar backend y frontend asegurándose que funcione.

Pedir al agente de IA que genere documentos .md (al estilo de los README.md) tanto para el front como el back, donde explique las decisiones de arquitectura, frameworks, librerías y demás que componen la solución.